Stolen keys are worthless: placeholder credentials that become real only inside encryption, with tripwires

Spencer Nunamaker
junovhs@gmail.com · https://github.com/junovhs/keyhandoff

Abstract

Developers keep API keys where their tools can read them: in `.env` files, in environment variables, in shell history and logs. Supply-chain malware and, increasingly, AI coding agents read the same places. Vaults and run wrappers (`op run`, `doppler run`) keep keys out of files but hand the real key to the program, where anything in its process can take it. Placeholder proxies keep the real key out of the program but require routing traffic through a man-in-the-middle and only speak HTTP.

We present a design in which a program never holds the key it uses. It holds a placeholder with the key's prefix and length. A hook in the program's own TLS library swaps the real key in at the moment a write is encrypted, and only on a connection whose certificate chain the hook has verified itself, against the system's roots, as one of the key's servers. Because nothing legitimate sends a placeholder anywhere else, every placeholder is also a tripwire. A write carrying one elsewhere, as written or in common encodings, is refused before a byte leaves. The run that tried is named, the destination is refused from then on, and a signed warning reaches teammates through a folder they already share. Each run gets fresh placeholders, so one found later traces back to its run. We implement the design in KeyHandoff, a small desktop key manager.

We evaluate coverage across 15 client and protocol combinations, 11 reconstructed attacks drawn from documented supply-chain campaigns and AI-agent misuse, overhead, and an adversarial review.

- **Coverage:** 12 of the 15 combinations are protected, including HTTP/2, where we rewrite HPACK-compressed headers [17]. The other three never release the key.
- **Attacks:** in every reconstructed attack the attacking code held only a placeholder, and no real key left the machine. Every send was stopped and attributed.
- **Overhead:** a swapped request costs about 0.26 ms; anything else costs about 2 µs.
- **Adversarial review:** it found, and we fixed, a way for in-process code to obtain the key through the hook's own token. It also states precisely what still does: code that targets the hook in memory, and use of the key at its own server.

1. The problem

A key is safe until a program needs it. At that point it ends up somewhere other code can read:

- **Files and the environment.** `.env` files, shell profiles and CI variables hold keys in plain text. Stealers target exactly these. The August 2025 compromise of `nx` packages inventoried

environment variables and `.env` files, asked locally installed AI command-line tools to search for more, and uploaded the haul triple-base64-encoded to a public GitHub repository created in the victim's own account [1, 2]. The September 2025 Shai-Hulud worm, run from npm install scripts, harvested environment variables and cloud keys, published them double-base64-encoded to a public repository named `Shai-Hulud`, and planted a GitHub Action that sent each repository's secrets to a webhook [3].

- **AI coding agents.** An agent with shell access can print, encode or upload a key. It may do so because it was asked, or because it was told to by text it read: an issue, a web page, a README. An agent can also run the wrapper itself (`op run -- sh -c 'env'`).
- **Logs and transcripts.** Keys printed while debugging end up in terminal scrollback, CI logs and agent transcripts.

Existing tools each remove part of this exposure:

Approach	Key in a file	Key in the program	Traffic	Protocols
`.env`	yes	yes	direct	any
Vault + run wrapper (`op run`, `doppler run`, `infisical run`) [4–6]	no	yes	direct	any
Placeholder proxy (agent sandboxes, `agent-vault`, `iron-proxy`, fnox, GhostKey) [7–11]	no	no	through a MITM proxy and its CA	HTTP
Kernel uprobe swap (Kloak) [14]	no	no	direct	TLS, in a cluster, with privileges
This work	no	no (except during the swap)	direct	anything over TLS the hook covers

2. Threat model

We defend a developer's own computer, which is not compromised at the administrator level. The attacker runs code as the developer, either inside a program the developer starts (a malicious dependency, an agent's command) or beside it. That code can read the program's environment, files and output, and can make network connections. It cannot change the kernel, read other users' memory, or break TLS.

The goal is that the key never leaves the machine except to its own server, inside a TLS connection whose certificate the hook verified. Everything the attacker can take is a placeholder. Placeholders die when their run ends, and every attempt to send one elsewhere is visible.

Out of scope, and said plainly in `docs/threats.md`:

- **Use at the key's own server.** Code in the program can make requests to the key's own server as the developer, exactly as any program allowed the key could.
- **Code that targets the hook in memory.** The key is in the program's memory for the duration of each swap.
- **Administrator-level compromise.**

3. Design

The design has four parts. A **key service**, a long-running process of the developer's, holds the keys and decides which programs may use them. A **runner** starts a program with its keys. A **hook**, loaded into that program, swaps keys in and watches for placeholders leaving. **Team folders**, ordinary folders teammates already share, carry signed warnings between their key services. In our prototype, KeyHandoff is the key service, and its `keyhandoff run` command is the runner.

3.1 Running a program with keys

The runner reads pointers to keys (in our prototype, lines like `API_KEY=handoff:Name`) from `.env` and the environment. It asks the key service for those keys over a per-user local socket, naming the exact command it will start, then starts the command.

Approval is bound to the command. An approval covers the program's path, its arguments, and the SHA-256 of every file the arguments name. An edited script, or the same interpreter running other code (`python -c …`), has to ask again. The runner's claim about the command is believed only when the request comes from the runner's own executable, which the key service checks by file identity or content.

Prompts are AI-aware. The key service walks the requesting process's parent chain. If Claude Code, Cursor, Codex, Copilot, Aider, Windsurf or Gemini CLI appears in it, saved approvals don't count: the prompt names the agent and the exact command, and offers "Allow for 30 minutes", which is never saved, instead of "Always". Name-based detection is a warning; the binding to the command is the guarantee.

Output is masked. The program's stdout and stderr, and those of everything it starts, pass through a masker. It hides the key or placeholder printed plain, base64 (at each byte offset), hex, percent-encoded or reversed, and any run of 12 or more of those characters, across chunk boundaries.

3.2 Placeholders

Each key records the servers it is for. They are guessed from well-known prefixes (`sk_live_` means `api.stripe.com`, `sk-ant-` means `api.anthropic.com`) and can be edited. For such a key, the program gets a placeholder instead of the key: the same prefix and length, with random characters of the same classes after it, fresh for every run. Same length means client libraries accept it and `Content-Length` doesn't change. A key its provider signs with rather than sends (AWS) gets no default server, because a placeholder cannot work for it.

3.3 The swap

On Linux the runner preloads a small C library into the program (`LD_PRELOAD`). It interposes OpenSSL's `SSL_write` and `SSL_write_ex`, resolving OpenSSL at run time, so the library needs no

headers and works with OpenSSL 1.1 and 3. When a write holds a placeholder, the hook does four things:

1. **Verify the chain itself.** It checks the peer's certificate chain against the system's default roots and, if set, the key's own CA. It never uses the program's trust settings, `SSL_CERT_FILE` or the verification result the program saw, because code in the program can change all three.
2. **Check the host.** It checks the leaf certificate against the key's hosts with `X509_check_host`. The name asked for in SNI and the `Host` header play no part.
3. **Fetch the key.** It asks the key service for the key, presenting the run's token. The key service answers only processes descended from the run's runner, and only while the runner is connected.
4. **Swap.** It writes the key into a private copy of the buffer, writes that copy (with `SSL_MODE_ACCEPT_MOVING_WRITE_BUFFER`, so a retried write can come from another buffer), and wipes it.

The verification verdict is cached per connection for a minute. The cache key is the connection object together with the leaf certificate's signature bytes, so a reused object talking to another server misses.

HTTP/2. HPACK [17] compresses headers, usually with a static Huffman code, before they reach `SSL_write`, so a placeholder in an HTTP/2 header is never there as written. On a connection that negotiated `h2`, the hook follows the frames from the connection preface. It decodes each complete HEADERS block's string literals, swaps the placeholder where the connection is verified, and re-encodes each string with the same representation. Because the key is as long as its placeholder, the dynamic tables on both sides stay in step. The Huffman table was not typed in. We measured each symbol's code length from the system's libnghttp2 encoder, rebuilt the canonical code from those lengths, and checked it byte for byte against libnghttp2 on 1,898 random strings; the generator is part of the artifact (§9).

HTTP Basic auth (`curl -u key:`) sends the key base64-encoded. At the key's own server, the hook decodes the base64 run, swaps, and encodes it again at the same length.

Node.js links its own OpenSSL, out of the preloaded hook's reach. The runner adds a `--require` module to `NODE_OPTIONS`. It wraps the TLS socket write path, waits for the handshake, verifies the chain itself (with `crypto.X509Certificate`, against Node's root store and the key's CA), and swaps or refuses.

The token. The hook reads its token at load and overwrites it in place. Just unsetting it would leave it in `/proc/self/environ` (§6.4). It passes the token on to programs the hook starts through `exec`, `posix_spawn`, `system` and `popen`.

3.4 Tripwires

A placeholder headed anywhere but its key's servers is a leak in the act.

What is refused. The hook refuses, before any of it leaves:

- a TLS write holding a placeholder on a connection not verified for its key;
- any write holding one on a plain internet socket (`send`, `sendto`, `sendmsg`, `write`, `writev`);
- a name lookup (`getaddrinfo`) containing one.

It recognises the placeholder as written, in standard base64 nested up to three levels deep at every

byte offset, in URL-safe base64, hex, percent-encoding and reversed. Those 47 patterns are scanned in one pass over a hashed set of 4-byte windows, probed at a stride of the shortest pattern length minus three.

What happens next. The key service keeps an alert naming the key, the command that tried, and the destination. It refuses that destination for later runs: by name, by certificate name, and by address. It also writes a signed warning into each team folder the developer is on. Teammates' key services refuse destinations from warnings signed by a member of the team.

Lineage. Every run's placeholders are recorded, so a placeholder found later says which key, command and time it came from, even base64-encoded up to three times. A placeholder can turn up in a log, a commit or an agent transcript.

4. Implementation

KeyHandoff is a Tauri app with a Rust core and React screens. It is both the key service and, as `keyhandoff run`, the runner. The parts described here are:

- **Runner:** `run.rs`, and `mask.rs` for output masking.
- **Placeholders and hosts:** `placeholder.rs`.
- **The C hook:** `src-tauri/hook/keyhandoff_tls.c`, about 1,400 lines, with its HPACK Huffman table generated by `gen_hpack_huffman.py`, compiled by `build.rs` and embedded in the executable.
- **The Node hook:** `keyhandoff_node.cjs`, about 210 lines.
- **Signed team warnings:** a new object kind in the sealing crate, `crates/seal/src/warning.rs`.

The hook is written out on first use to a private folder under `$XDG_RUNTIME_DIR`, named by its contents.

5. Evaluation

Everything below is regenerated by `eval/run.sh` and, for coverage and attacks, by CI on every push. CI runs on GitHub's Ubuntu 22.04 runner; the Windows build and tests also run there, but no Windows cell is claimed. The numbers below are from a Ryzen 9 3900X running Pop!_OS 22.04 with OpenSSL 3.0.2.

5.1 Coverage

The same keyed request goes to a server whose certificate is valid for the key's host, and to a stranger with a valid certificate for another name. Both servers use the same test CA, configured as the key's CA.

Client	TLS	Protocol	Key's server got	Stranger got
Python http.client, requests, httpx	system OpenSSL	HTTP/1.1, header	key	nothing
Python	system OpenSSL	HTTP/1.1, chunked body	key	nothing

Client	TLS	Protocol	Key's server got	Stranger got
Python	system OpenSSL	HTTP/1.1, gzip body	placeholder	placeholder
Python	system OpenSSL	WebSocket upgrade header	key	nothing
Python	system OpenSSL	WebSocket message (masked)	placeholder	placeholder
curl	system OpenSSL	HTTP/1.1 header; Basic auth	key	nothing
curl	system OpenSSL, nghttp2	HTTP/2 header (HPACK)	key	nothing
curl	system OpenSSL, nghttp2	HTTP/2 body	key	the request, without key or placeholder
curl	none	plain HTTP to a stranger	–	nothing
Ruby Net::HTTP	system OpenSSL	HTTP/1.1, header	key	nothing
Node.js https	Node's OpenSSL	HTTP/1.1, header	key	nothing
Go net/http	Go crypto/tls	HTTP/1.1, header	placeholder	placeholder

Twelve of the 15 cells are protected. The three that aren't (gzip bodies, masked WebSocket messages, Go) never release the key: the request to the key's server fails, and the stranger gets a placeholder. Not tested here: Postgres and Redis over TLS, Java, rustls, HTTP/3, and Windows.

5.2 Attack replays

These are reconstructions of documented behaviour, not live malware. The Shai-Hulud replays send the double-base64 haul to a webhook, folding the worm's two channels (a public repository and a webhook) into one. They run locally against a synthetic key; `eval/replays/README.md` has the ethics note.

Replay	Attacker held	Left the machine	Stopped and attributed
Nx "s1ngularity": env, `.env`, AI-CLI inventory, triple base64, GitHub-style upload	placeholder	nothing	yes

Replay	Attacker held	Left the machine	Stopped and attributed
Shai-Hulud: env, double base64, HTTPS webhook (Node)	placeholder	nothing	yes
Shai-Hulud to a plain HTTP webhook	placeholder (masked)	nothing	yes
`.env` hunter, posted in clear	placeholder	nothing	yes
Agent (under a parent named `claude`): print / base64 the key	masked output	–	– (nothing sent)
Agent: `env | curl` to a paste site	masked output	nothing	yes
Agent: post to a webhook	masked output	nothing	yes
Agent: key in a DNS name	masked output	–	yes
Agent: write the key into a committed file	placeholder	–	– (traced to its run)
Use the key at its own server	–	the request, as you	no (out of scope)

Afterwards, a teammate on the same team folder was refused every stopped destination. The loot from the Nx, Shai-Hulud, `.env` hunter and commit replays traced back to the run that produced it.

5.3 Overhead

A Python client keeps one TLS connection open to a local server. It sends 2,000 small requests after 50 warm-up ones, then uploads 64 MiB in 64 KiB writes. Figures are the median of 9 alternating rounds. The baseline ("direct") is started the same way: detached, through the runner, with no hook.

	Per request	Upload
Direct	61–63 µs	~940 MiB/s
Hook, nothing to swap	+2 µs	within noise (−3%)
Hook, key swapped in every request	+262 µs	within noise (−13% against a ±20% spread)

The swap's cost is one local round trip to the key service for the key; the chain check is cached per connection. Measuring forced two fixes:

- **Uploads first ran 80% slower** under the hook, because 47 patterns were each searched separately, on both the plaintext and the ciphertext. The single-pass hashed scan fixed that.

- **A swapped request first cost about 730 µs,** because the chain was verified every time and the parent chain was walked on every connection.

We have not yet compared `op run` or a placeholder proxy, and the planned week of ordinary development under the hook has not been run.

5.4 Adversarial review

`docs/threats.md` tabulates 21 attacks by an attacker running as the user. Each is classed as stopped, worthless (only a placeholder leaves), gets the key, or out of scope, with a test or a reason.

The review found one real hole, now fixed. The hook removed its token with `unsetenv`, but the token stayed in the process's first environment block, readable at `/proc/self/environ`. Code in the program could read it and ask the key service for the key. The test that demonstrates this (`attacks_on_placeholders_get_no_key`) fails without the fix.

These attacks still get the key, by design or as stated limits:

- **A token in a child without the hook.** A statically linked binary's environment keeps the token, and in-run code could read it there.
- **In-process memory reads** during a swap.
- **Use at the key's own server.**

No one outside the project has reviewed the table yet.

6. Discussion

6.1 What placeholders change

The common theft paths become harmless: reading the environment or `.env`, printing, uploading, pasting into an agent transcript, committing. Each yields a placeholder that dies with its run, and every send of one through a covered path is stopped and attributed.

What remains open is narrower and different in kind. The attacker must either use the key through its own legitimate server, which is visible in that provider's logs, or write code that targets the hook specifically.

6.2 Why the hook, and not a proxy

A proxy sees traffic only if the program routes it there and trusts the proxy's CA. Code in the program can undo both. It also understands only the protocols the proxy parses.

The in-process hook works on any protocol over the TLS libraries it covers, with no routing and no new CA. It decides on the certificate the server actually presented, not on what the program asked for.

The cost is that the key is in the program's memory during the swap. Moving the swap out of the process, with an eBPF uprobe on `SSL_write` as Kloak does in Kubernetes [14], would close that gap. It needs privileges a developer's desktop tool shouldn't ask for.

6.3 Why tripwires have no false positives

A placeholder is never a valid credential anywhere. Nothing legitimate can send it to a server other than its key's. A refused write is therefore always a real attempt.

The exceptions are configuration errors. A wrong host on the key makes legitimate requests fail closed rather than leak. A teammate's warning about a service the developer needs blocks it; warnings name their signer.

6.4 Limitations

- **Coverage.** Coverage is the TLS libraries hooked: dynamically linked OpenSSL and Node. Windows (schannel) is not built.
- **Placeholders that aren't seen.** Placeholders inside compression or other encodings aren't recognised, and neither are header blocks split across HTTP/2 CONTINUATION frames.
- **Agent detection** is by process name.
- **Evaluation.** The replays are reconstructions, run on one machine rather than in disposable VMs. The false-alarm week hasn't happened.

7. Related work

- **Run wrappers.** 1Password's `op run` [4], `doppler run` [5] and `infisical run` [6] inject keys into a child's environment. `op run` also masks secrets the child prints. The child holds the real key.
- **Placeholder proxies.** Docker's agent sandboxes [7], Infisical's Agent Vault [8], iron-proxy [9], fnox's proxy [10] and GhostKey [11] give the workload a placeholder and swap in the real key at an HTTP(S) proxy, which the workload must route through and whose CA it must trust. fnox notes that a same-user process can bypass the proxy's environment settings.
- **Sandbox credential brokers.** Bashkit's placeholder mode [12] swaps a placeholder in outbound HTTP headers for scripts it runs, and OpenSandbox's credential vault [13] keeps the real key in an egress sidecar outside the sandbox. A design proposed for Sandlock [15] keeps the secret in a supervisor process and gives the child a per-session phantom token. All of these protect a sandboxed guest; ours protects programs a developer runs on their own machine.
- **Kernel uprobes.** Kloak [14] swaps placeholders for Kubernetes workloads with an eBPF uprobe on `SSL_write`, restricted to allowed destinations. It is the closest mechanism to ours, at a different layer and for a different user.
- **eBPF TLS tracing.** Tools such as eCapture [16] read `SSL_write` plaintext through uprobes without changing it.
- **Honeytokens.** Decoy credentials that alarm when used [18] are the ancestor of our tripwires, and per-site unique decoys can show where a leak happened [19]. Honeywords [20] plant decoy passwords beside the real one. Our placeholders differ in that they are the credential the program actually uses, fresh per run.
- **Artificial immune systems.** Forrest, Hofmeyr and others applied self/non-self discrimination to intrusion detection [21, 22].

8. Ethics

The replays reimplement only the collection and exfiltration steps described in public reports. They have no propagation, persistence or destructive behaviour. They run against local servers with synthetic keys, and `.test` names never resolve outside the machine.

9. Artifact

Run `eval/run.sh` from a checkout. It needs Rust, Node.js, Python 3, openssl, curl and setsid; Ruby, Go, `requests` and `httpx` add their rows when present.

It writes `eval/results/coverage.md`, `replays.md` and `benchmarks.md`, with JSON beside each. CI (`.github/workflows/check.yml`) reruns the coverage matrix and the replays on every push, and fails if a claimed cell isn't protected, a stranger ever gets a key, or a replayed send isn't stopped and attributed.

Acknowledgments

Claude (Anthropic), an AI model, was used extensively to write the software, run the evaluation and draft this paper. The author reviewed the text, checked every number against `eval/results/` and every reference against its source, and takes responsibility for the contents.

References

[1] Wiz Research. s1ngularity: supply chain attack leaks secrets on GitHub: everything you need to know. August 2025. https://www.wiz.io/blog/s1ngularity-supply-chain-attack

[2] Wiz Research. s1ngularity's aftermath: AI, TTPs, and impact in the Nx supply chain attack. 2025. https://www.wiz.io/blog/s1ngularitys-aftermath

[3] Wiz Research. Shai-Hulud: ongoing package supply chain worm delivering data-stealing malware. September 2025. https://www.wiz.io/blog/shai-hulud-npm-supply-chain-attack

[4] 1Password. op run (CLI reference). https://www.1password.dev/cli/reference/commands/run

[5] Doppler. CLI guide. https://docs.doppler.com/docs/cli

[6] Infisical. infisical run (CLI reference). https://infisical.com/docs/cli/commands/run

[7] Docker. Docker Sandboxes: credentials. https://docs.docker.com/ai/sandboxes/security/credentials/

[8] Infisical. Agent Vault. https://github.com/Infisical/agent-vault

[9] ironsh. iron-proxy. https://github.com/ironsh/iron-proxy

[10] fnox. Proxy guide. https://fnox.jdx.dev/guide/proxy.html

[11] GhostKey. https://github.com/jhaji2911/GhostKey

[12] Bashkit. Credential injection. https://docs.rs/crate/bashkit/latest/source/docs/credential-injection.md

[13] OpenSandbox. Credential vault. https://github.com/opensandbox-group/opensandbox

[14] spinningfactory. Kloak. https://github.com/spinningfactory/kloak

[15] multikernel. RFC: credential injection via the proxy (Sandlock issue 66). https://github.com/multikernel/sandlock/issues/66

[16] gojue. eCapture. https://github.com/gojue/ecapture

[17] R. Peon and H. Ruellan. HPACK: Header Compression for HTTP/2. RFC 7541, IETF, 2015.

[18] L. Spitzner. Honeytokens: the other honeypot. SecurityFocus, 2003.

[19] J. DeBlasio, S. Savage, G. M. Voelker and A. C. Snoeren. Tripwire: inferring internet site compromise. In Proc. ACM Internet Measurement Conference (IMC), 2017.

[20] A. Juels and R. L. Rivest. Honeywords: making password-cracking detectable. In Proc. ACM CCS, 2013.

[21] S. Forrest, S. A. Hofmeyr, A. Somayaji and T. A. Longstaff. A sense of self for Unix processes. In Proc. IEEE Symposium on Security and Privacy, 1996.

[22] S. A. Hofmeyr and S. Forrest. Architecture for an artificial immune system. Evolutionary Computation 8(4), 2000.